Collectives Variable Data and Non-blocking Communication

Lecture 18
March 23, 2026

Revision

```
int N = atoi (argv[1]);
int column = atoi (argv[2]);
int count = atoi (argv[3]);
int blocklen = atoi (argv[4]);
int stride = atoi (argv[5]);
int data[N][N], received[N*blocklen];
```

0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

```
class $ mpirun -np 2 ./vector2D 4 0 4 1 4

0 1 2 3

class $ mpirun -np 2 ./vector2D 4 0 4 2 4

0 1 1 2 2 3 3 4
```

0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

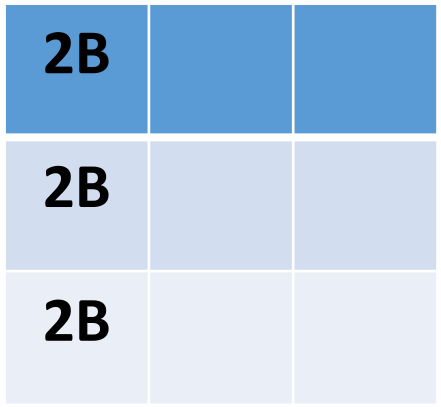
MPI Collectives
Variable Data Versions

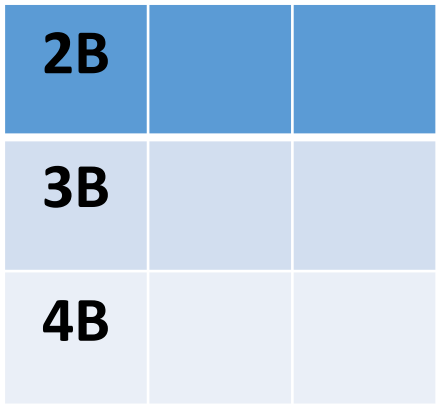
P0	P1	P2*	P3	Function	P0	P1	P2	P3
a	b	c	d	MPI_Gather			a,b,c,d	
a	b	c	d	MPI_Allgather	a,b,c,d	a,b,c,d	a,b,c,d	a,b,c,d
		a,b,c,d		MPI_Scatter	a	b	c	d
a,b,c,d	e,f,g,h	i,j,k,l	m,n,o,p	MPI_AlltoAll	a,e,i,m	b,f,j,n	c,g,k,o	d,h,l,p
		b		MPI_Bcast	b	b	b	b
SBuf	SBuf	SBuf	SBuf		RBuf	RBuf	RBuf	RBuf

Credit: IITKgp

Collectives – Variable Data

- Communicate unequal amount of data to/from each process involved in the collective function call



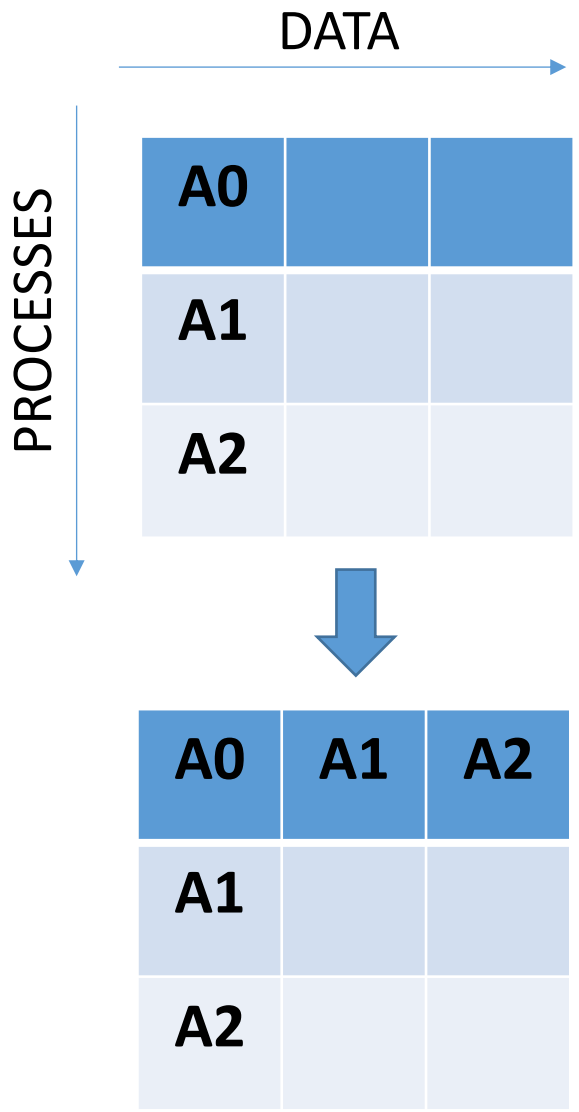


MPI_Gather – Recap

- Gathers values from all processes to a root process
- int MPI_Gather (sendbuf, sendcount, sendtype, *recvbuf*, recvcount, recvtype, root, comm)
- Arguments recv* not relevant on non-root processes
- recvcount → size of any (single) receive
- Distinct values (may be same) received from non-root processes at the root process
- Example: Reading multiple files (1 file per process)

Q: Equivalent point-to-point communications for the same?

- MPI_Recv at root
- MPI_Send at non-root

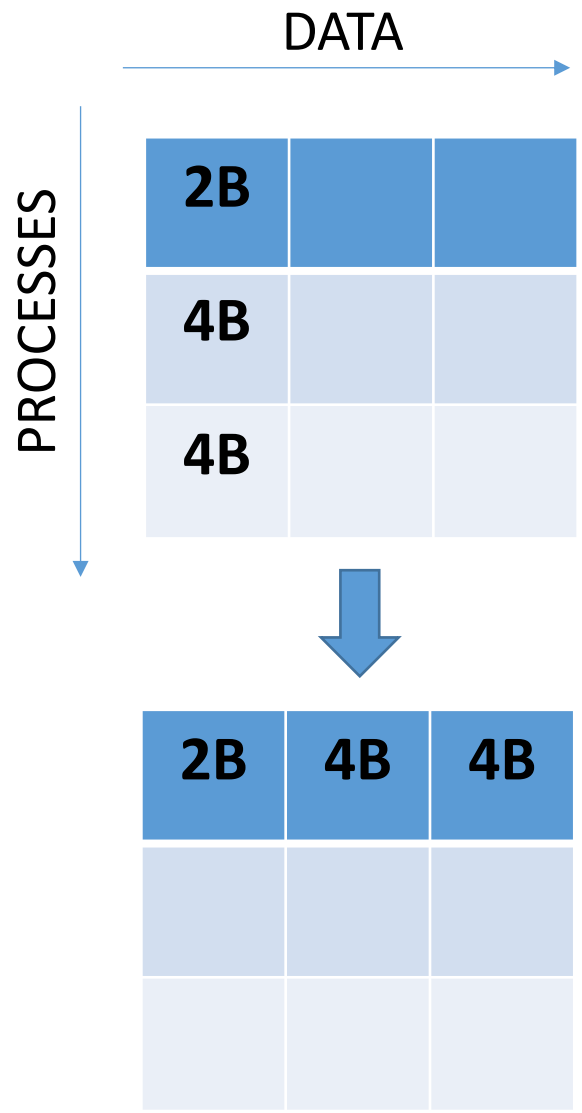


MPI_Gatherv

- Root gathers different amounts of data from the other processes
- int MPI_Gatherv (sendbuf, sendcount, sendtype, *recvbuf*, recvcounts, displs, recvtype, root, comm)
- recvcounts – Number of elements to be received from each process
- displs – Displacement at which to place the received data

MPI_Recv (recvbuf+displs[i], recvcounts[i], recvtype, i, i, comm, &status) at root for i^{th} process

MPI_Send at non-root



```
int MPI_Gatherv (

const void *sendbuf,
int sendcount,
MPI_Datatype sendtype,
void *recvbuf,
const int recvcounts[],
const int displs[],
MPI_Datatype recvtype,
int root,
MPI_Comm comm

)
```

https://www.mpich.org/static/docs/v4.1/www3/MPI_Gatherv.html

Input Parameters

sendbuf
starting address of send buffer (choice)

sendcount
number of elements in send buffer (non-negative integer)

sendtype
data type of send buffer elements (handle)

recvcounts
non-negative integer array (of length group size) containing the number of elements that are received from each process (non-negative integer)

displs
integer array (of length group size). Entry i specifies the displacement relative to recvbuf at which to place the incoming data from process i (integer)

recvtype
data type of recv buffer elements (handle)

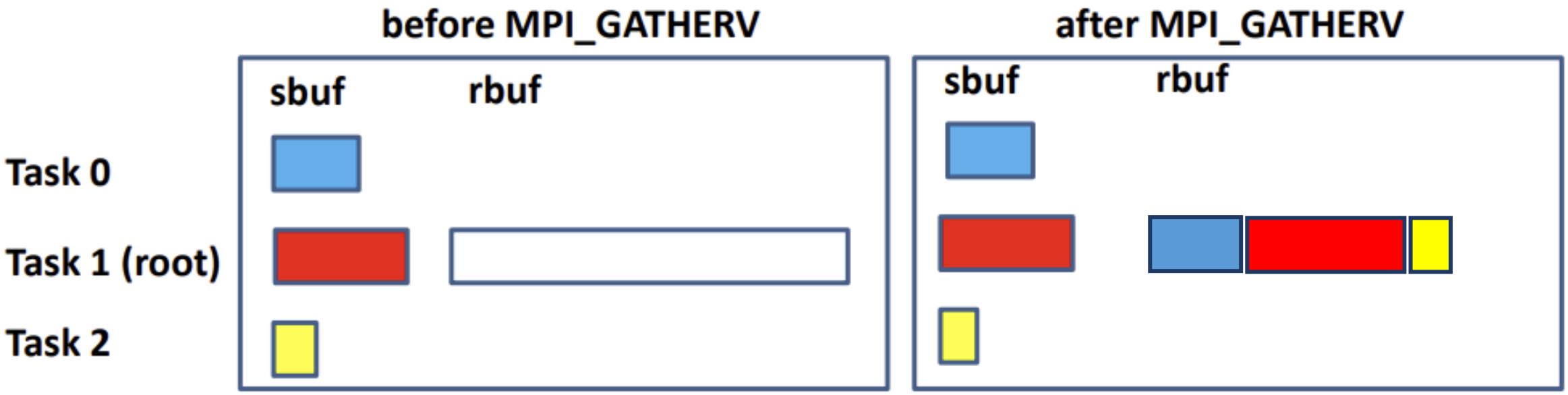
root
rank of receiving process (integer)

comm
communicator (handle)

Output Parameters

recvbuf
address of receive buffer (choice)

MPI_Gatherv Illustration



MPI_Gatherv Code

```
// Allocate message at the senders (may be different sized for different processes)
int arrSize = atoi(argv[1])*(rank+1);  // synthetically creating different array sizes
int message[arrSize];
int countArray[numtasks], displArray[numtasks];
```

```
int message[arrSize];
int countArray[numtasks], displArray[numtasks];

int displ = 0;                          // note that root process is 0 here

// this information is needed by the root
if (!rank)
 for (i = 0; i < numtasks; i++) {
  countArray[i] = arrSize*(i+1);          // depends on the counts, root may need to get it from the processes
  displArray[i] = displ;
  displ += countArray[i];
  printf ("%d %d %d\n", i, countArray[i], displArray[i]);
 }
```

```
int message[arrSize];
int countArray[numtasks], displArray[numtasks];

int displ = 0;                        // note that root process is 0 here

// this information is needed by the root
if (!rank)
 for (i = 0; i < numtasks; i++) {
  countArray[i] = arrSize*(i+1);          // depends on the counts, root may need to get it from the processes
  displArray[i] = displ;
  displ += countArray[i];
  printf ("%d %d %d\n", i, countArray[i], displArray[i]);
 }

if (!rank)
  printf ("\n");

// every process initializes their local array
srand(time(NULL));
for (i = 0; i < arrSize; i++) {
  message[i] = i; // (double)rand() / (double)RAND_MAX;
}
```

```c
int message[arrSize];
int countArray[numtasks], displArray[numtasks];

int displ = 0;                        // note that root process is 0 here

// this information is needed by the root
if (!rank)
 for (i = 0; i < numtasks; i++) {
  countArray[i] = arrSize*(i+1);          // depends on the counts, root may need to get it from the processes
  displArray[i] = displ;
  displ += countArray[i];
  printf ("%d %d %d\n", i, countArray[i], displArray[i]);
 }

if (!rank)
  printf ("\n");

// every process initializes their local array
srand(time(NULL));
for (i = 0; i < arrSize; i++) {
  message[i] = i; // (double)rand() / (double)RAND_MAX;
}

int recvMessage[displ];    // significant at the root process

// receive different counts of elements from different processes
MPI_Gatherv (message, arrSize, MPI_INT, recvMessage, countArray, displArray, MPI_INT, 0, MPI_COMM_WORLD);

if (!rank)
 for (i = 0; i < displ; i++) {
   printf ("%d %d\n", i, recvMessage[i]);
 }
```

Demo

```
class $ mpirun -np 2 ./gatherv 2
0 2 0
1 4 2

0 0
1 1
2 0
3 1
4 2
5 3
```

#ranks

arrSize

Demo

```
class $ mpirun -np 3 ./gatherv 2
0 2 0
1 4 2
2 6 6

0 0
1 1
2 0
3 1
4 2
5 3
6 0
7 1
8 2
9 3
10 4
11 5
class $
```

arrSize

Bug in one of the data size related arguments

```
Fatal error in PMPI_Gatherv: Message truncated, error stack:
PMPI_Gatherv(435).....................: MPI_Gatherv failed(sbuf=0x7ffd379c4250, scount=2, MPI_INT, rbuf=0x7ffd379c4210, rcnts=0x7ffd379c4240,
 displs=0x7ffd379c4230, MPI_INT, root=0, MPI_COMM_WORLD) failed
MPIR_Gatherv_impl(235)................:
MPIR_Gatherv(151).....................:
MPIDI_CH3_PktHandler_EagerShortSend(363): Message from rank 1 and tag 4 truncated; 16 bytes received but buffer size is 8
class $ vi gatherv.c
class $ !mpicc
mpicc -o gatherv gatherv.c
class $ !mpirun
mpirun -np 2 ./gatherv 2
0 2 0
1 4 2
class $
```

```
class $ time mpirun -np 3 ./gatherv 2

real    0m0.007s
user    0m0.004s
sys     0m0.010s
class $ time mpirun -np 30 -hosts csews4:10,csews2:10,csews3:10 ./gatherv 200

real    0m0.695s
user    0m0.026s
sys     0m0.006s
class $ time mpirun -np 30 -hosts csews4:10,csews2:10,csews3:10 ./gatherv 20000
```

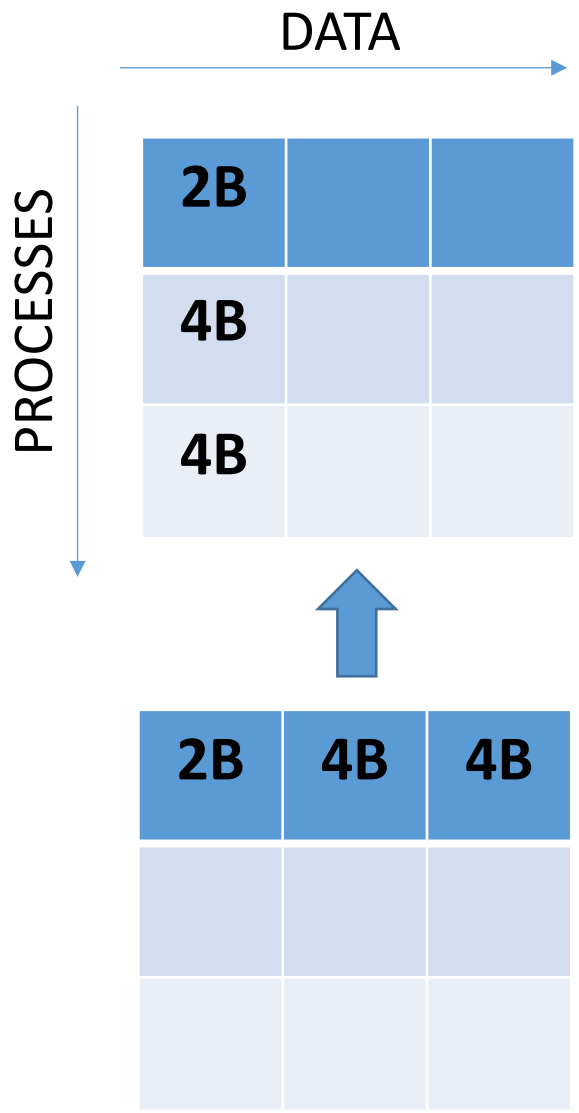
```
class $ time mpirun -np 3 ./gatherv 2

real    0m0.007s
user    0m0.004s
sys     0m0.010s
class $ time mpirun -np 30 -hosts csews4:10,csews2:10,csews3:10 ./gatherv 200

real    0m0.695s
user    0m0.026s
sys     0m0.006s
class $ time mpirun -np 30 -hosts csews4:10,csews2:10,csews3:10 ./gatherv 20000

real    0m1.857s
user    0m0.033s
sys     0m0.002s
class $ time mpirun -np 60 -hosts csews4:10,csews2:10,csews3:10,csews5:10,csews6:10,csews7:10 ./gatherv 20000
```

```
class $ time mpirun -np 3 ./gatherv 2

real    0m0.007s
user    0m0.004s
sys     0m0.010s
class $ time mpirun -np 30 -hosts csews4:10,csews2:10,csews3:10 ./gatherv 200

real    0m0.695s
user    0m0.026s
sys     0m0.006s
class $ time mpirun -np 30 -hosts csews4:10,csews2:10,csews3:10 ./gatherv 20000

real    0m1.857s
user    0m0.033s
sys     0m0.002s
class $ time mpirun -np 60 -hosts csews4:10,csews2:10,csews3:10,csews5:10,csews6:10,csews7:10 ./gatherv 20000

real    0m2.965s
user    0m12.571s
sys     0m1.559s
class $ time mpirun -np 60 -hosts csews4:10,csews2:10,csews3:10,csews5:10,csews6:10,csews7:10 ./gatherv 800000
```

```
class $ time mpirun -np 3 ./gatherv 2

real    0m0.007s
user    0m0.004s
sys     0m0.010s
class $ time mpirun -np 30 -hosts csews4:10,csews2:10,csews3:10 ./gatherv 200

real    0m0.695s
user    0m0.026s
sys     0m0.006s
class $ time mpirun -np 30 -hosts csews4:10,csews2:10,csews3:10 ./gatherv 20000

real    0m1.857s
user    0m0.033s
sys     0m0.002s
class $ time mpirun -np 60 -hosts csews4:10,csews2:10,csews3:10,csews5:10,csews6:10,csews7:10 ./gatherv 20000

real    0m2.965s
user    0m12.571s
sys     0m1.559s
class $ time mpirun -np 60 -hosts csews4:10,csews2:10,csews3:10,csews5:10,csews6:10,csews7:10 ./gatherv 800000

real    1m5.621s
user    7m2.519s
sys     1m2.687s
class $
```

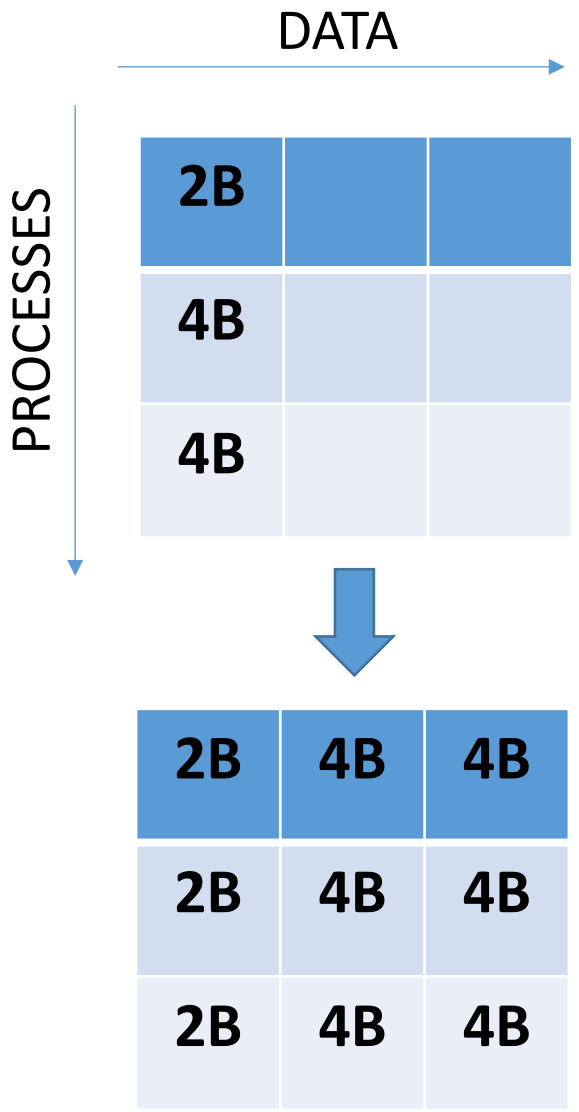
MPI_Scatterv

- Root scatters different amounts of data to the other processes
- int MPI_Scatterv (const void *sendbuf, const int *sendcounts, const int *displs, MPI_Datatype sendtype, void **recvbuf*, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm)
- sendcounts – Number of elements to be sent to each process
- displs – Displacement (relative to sendbuf) at which the data to be sent resides

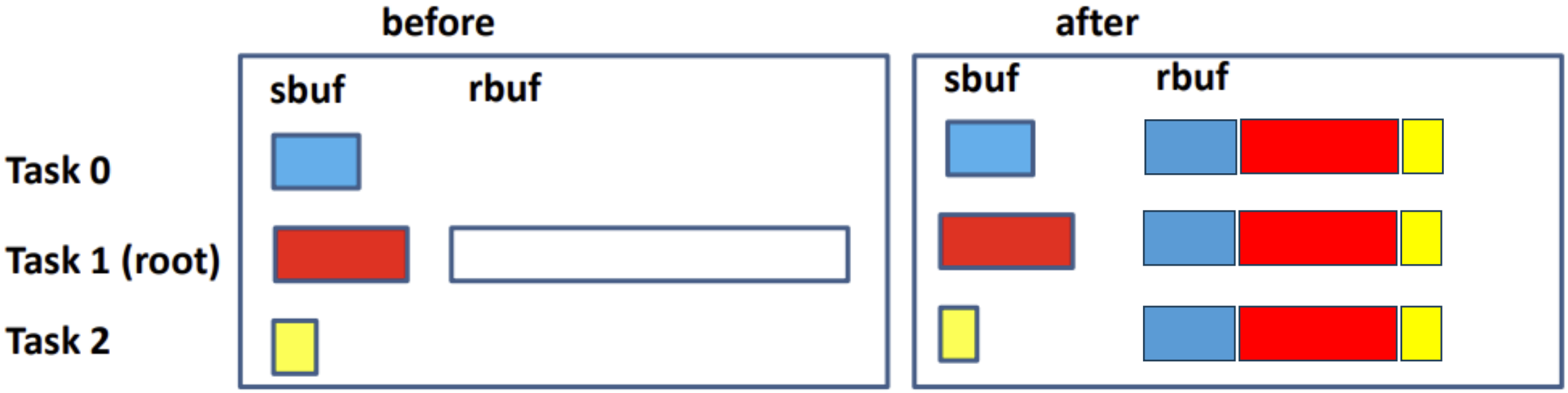


MPI_Allgatherv

- All processes gather values of different lengths from all processes
- int MPI_Allgatherv (sendbuf, sendcount, sendtype, *recvbuf*, recvcounts, displs, recvtype, comm)
- recvcounts – Number of elements to be received from each process
- displs – Displacement at which to place received data



MPI_AllGatherv Illustration



```
int message[arrSize];
int countArray[numtasks], displArray[numtasks];

int displ = 0;                          // note that root process is 0 here

// this information is needed by the root
if (!rank)
 for (i = 0; i < numtasks; i++) {
  countArray[i] = arrSize*(i+1);          // depends on the counts, root may need to get it from the processes
  displArray[i] = displ;
  displ += countArray[i];
  printf ("%d %d %d\n", i, countArray[i], displArray[i]);
 }

if (!rank)
  printf ("\n");

// every process initializes their local array
srand(time(NULL));
for (i = 0; i < arrSize; i++) {
  message[i] = i; // (double)rand() / (double)RAND_MAX;
}

int recvMessage[displ];    // significant at the root process

// receive different counts of elements from different processes
MPI_Gatherv (message, arrSize, MPI_INT, recvMessage, countArray, displArray, MPI_INT, 0, MPI_COMM_WORLD);

if (!rank)
 for (i = 0; i < displ; i++) {
   printf ("%d %d\n", i, recvMessage[i]);
 }
```

Modify the code to use MPI_Allgatherv

MPI_Allgatherv

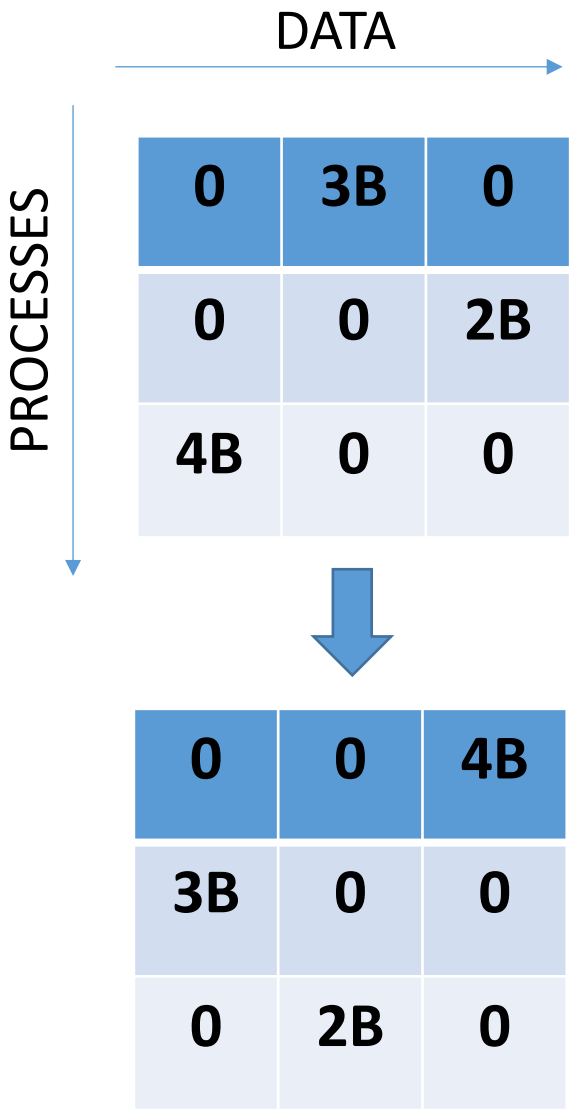
https://www.mpich.org/static/docs/v3.1/www3/MPI_Allgatherv.html

int MPI_Allgatherv (const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, const int *recvcounts, const int *displs, MPI_Datatype recvtype, MPI_Comm comm)

```
// receive different counts of elements from different processes
MPI_Allgatherv (message, arrSize, MPI_INT, recvMessage, countArray, displArray, MPI_INT, MPI_COMM_WORLD);
```

MPI_Alltoallv

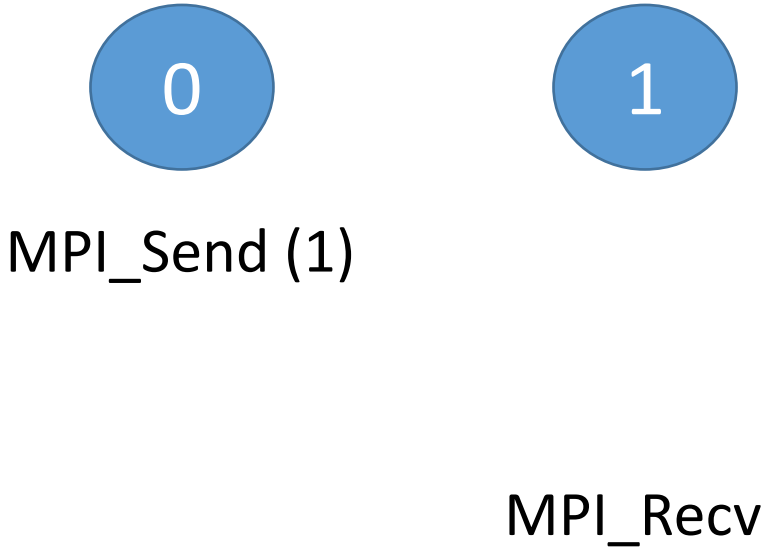
- Every process sends data of different lengths to other processes
- int MPI_Alltoallv (sendbuf, sendcount, sdispls, sendtype, *recvbuf*, recvcount, rdispls, recvtype, comm)
- Output parameter – recvbuf
- It’s not necessary to receive some data from all processes, i.e. some entries of count and displs may be 0



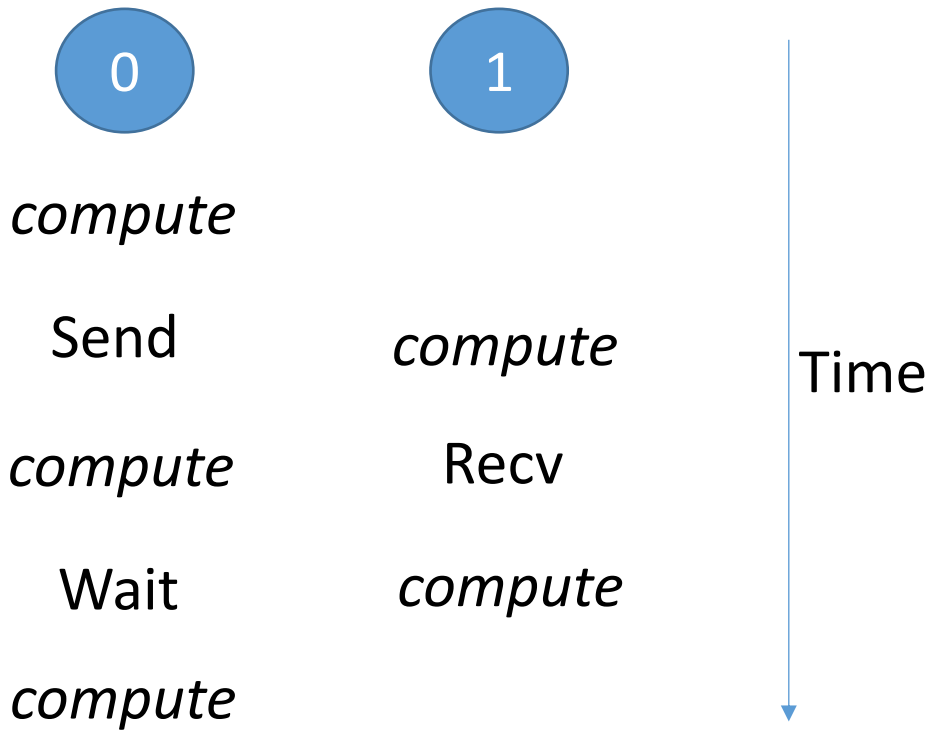
Non-blocking Communications

P2P Blocking – Performance Bottleneck

- MPI_Send (buf, count, datatype, dest, tag, comm)
- MPI_Recv (buf, count, datatype, source, tag, comm, status)



Computation Communication Overlap



Non-blocking Point-to-Point

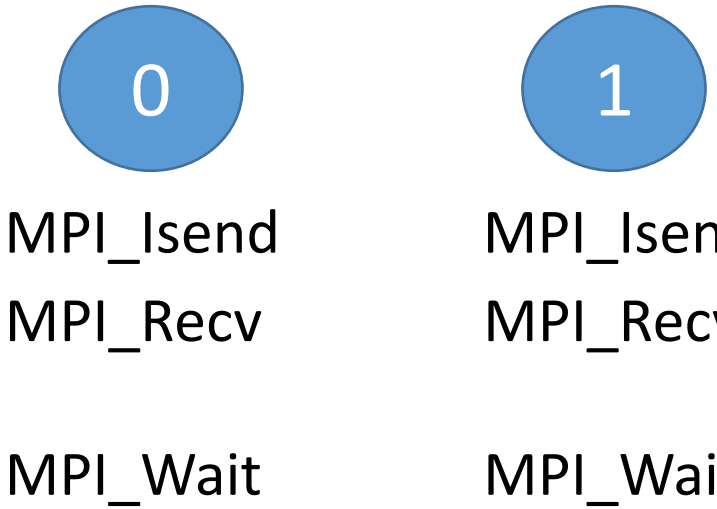
- MPI_Isend (buf, count, datatype, dest, tag, comm, request)
- MPI_Irecv (buf, count, datatype, source, tag, comm, request)

- MPI_Wait (request, status)
- MPI_Waitall (count, request, status)

Your code is incorrect if you do not use MPI_Wait or MPI_Waitall, after non-blocking communication.

Non-blocking Point-to-Point Safety

- MPI_Isend (buf, count, datatype, dest, tag, comm, request)
- MPI_Irecv (buf, count, datatype, source, tag, comm, request)
- MPI_Wait (request, status)



Safe

One-to-many Non-Blocking Communications

All ranks except 0 send to rank 0

```
// Write the code (H.W.)
// Q: What is missing? A: MPI_Wait
if I am rank 0
   for (r = 1; r<size; r++)
         MPI_Isend (to r)
else
         MPI_Recv (from 0)
```

Asynchronous Communication Progress

One to many (using MPI_Isend)

```
class $ mpirun -np 40 -f hostfile ./onetomanyisends 100000000
123.617642
class $ export MPIR_CVAR_ASYNC_PROGRESS=1
class $ mpirun -np 40 -f hostfile ./onetomanyisends 100000000
117.038334
class $ for i in `seq 1 3` ; do mpirun -np 40 -f hostfile ./onetomanyisends 100000000 ; done
116.899904
116.969498
116.891767
class $
```

Many-to-one Non-blocking P2P

```
// send from all ranks to the last rank
start_time = MPI_Wtime ();
if (myrank < size-1)
{
  MPI_Send(arr, BUFSIZE, MPI_INT, size-1, 99, MPI_COMM_WORLD);
}
else
{
```

```
}
time = MPI_Wtime () - start_time;

MPI_Reduce (&time, &max_time, 1, MPI_DOUBLE, MPI_MAX, size-1, MPI_COMM_WORLD);
if (myrank == size-1) printf ("Max time = %lf\n", max_time);
```

Many-to-one Non-blocking P2P

```
// send from all ranks to the last rank
start_time = MPI_Wtime ();
if (myrank < size-1)
{
  MPI_Send(arr, BUFSIZE, MPI_INT, size-1, 99, MPI_COMM_WORLD);
}
else
{
  int count, recvarr[size][BUFSIZE];
  for (int i=0; i<size-1; i++)
    MPI_Irecv(recvarr[i], BUFSIZE, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &request[i]);
  MPI_Waitall (size-1, request, status);
}
time = MPI_Wtime () - start_time;

MPI_Reduce (&time, &max_time, 1, MPI_DOUBLE, MPI_MAX, size-1, MPI_COMM_WORLD);
if (myrank == size-1) printf ("Max time = %lf\n", max_time);
```

Output

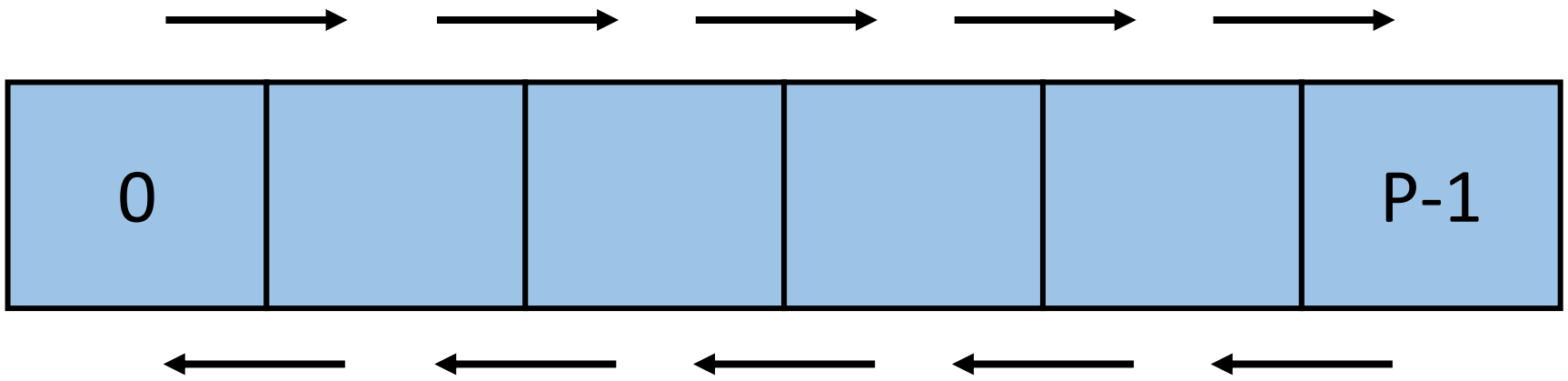
```
class $ for i in `seq 1 3` ; do mpirun -np 40 -f hostfile ./p2p 1000000 ; done
Max time = 1.395533
Max time = 1.278514
Max time = 1.352068
class $ for i in `seq 1 3` ; do mpirun -np 40 -f hostfile ./nb_p2p 1000000 ; done
Max time = 1.362506
Max time = 1.244787
Max time = 1.289337
```

```
class $ for i in `seq 1 3` ; do mpirun -np 40 -f hostfile ./p2p 100000000 ; done
Max time = 123.864696
Max time = 123.819141
Max time = 123.850547
class $ for i in `seq 1 3` ; do mpirun -np 40 -f hostfile ./nb_p2p 100000000 ; done
Max time = 122.848520
Max time = 122.804291
Max time = 122.797551
```

Why may you expect higher performance gap between the two when such applications are run on production supercomputers?

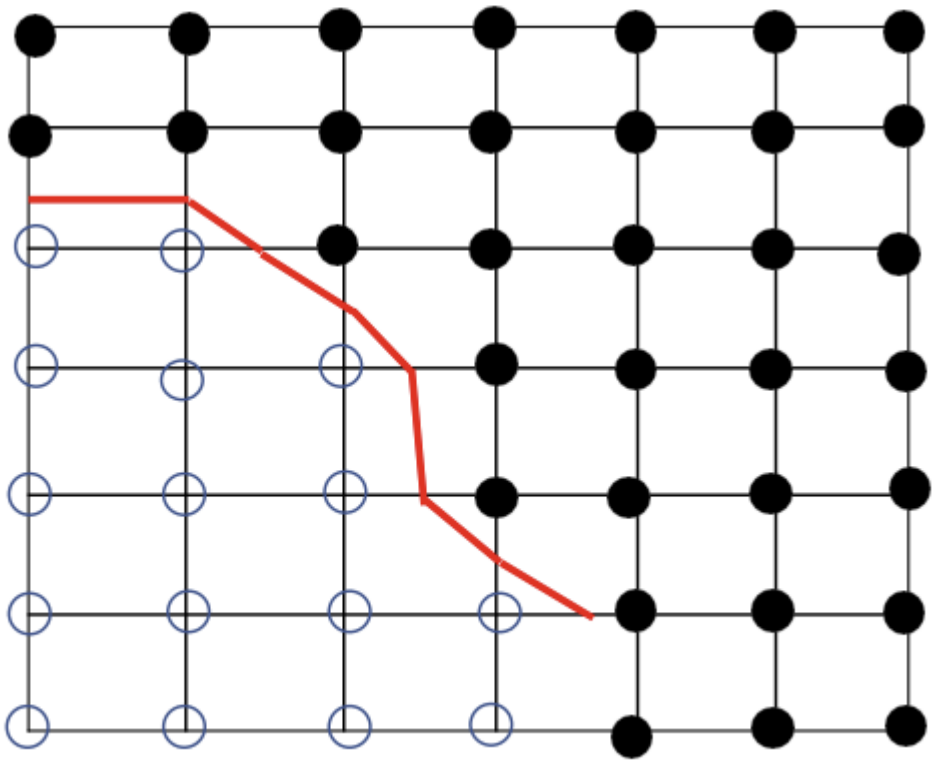
NN 1D using Non-blocking

H.W.: Revise the blocking solutions and write the code using non-blocking sends and receives



2D Isocontour Extraction: Marching Squares

- Given a 2D scalar field, compute isocontour (isoline) for isovalue = **C**
- This is usually done in a cell-by-cell manner using Marching Squares algorithm

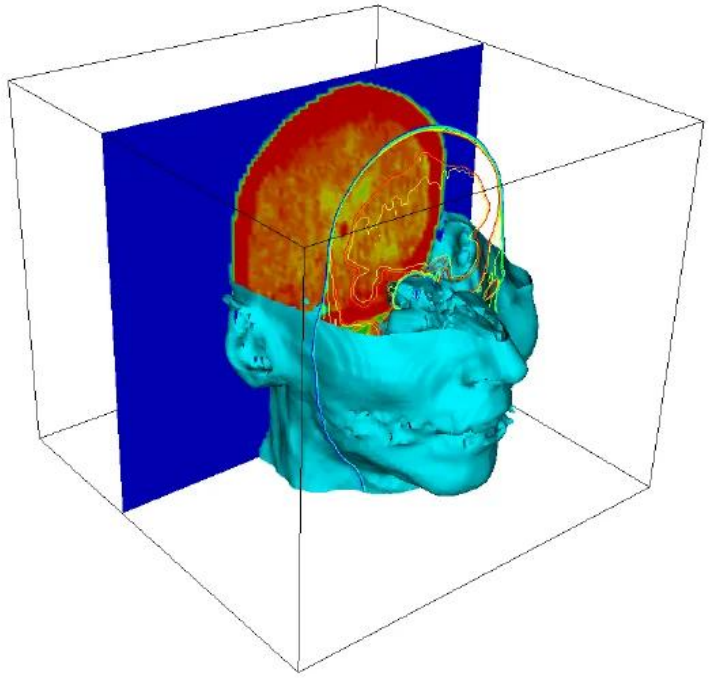


Contour value (isovalue) = C

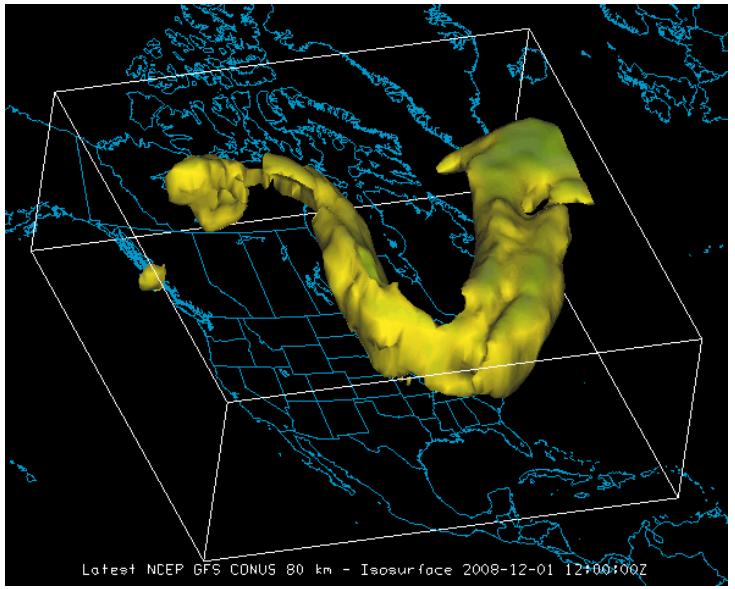
- ● Value > C
- ○ Value < C

Source: CS677: Topics in Large Data Analysis And Visualization

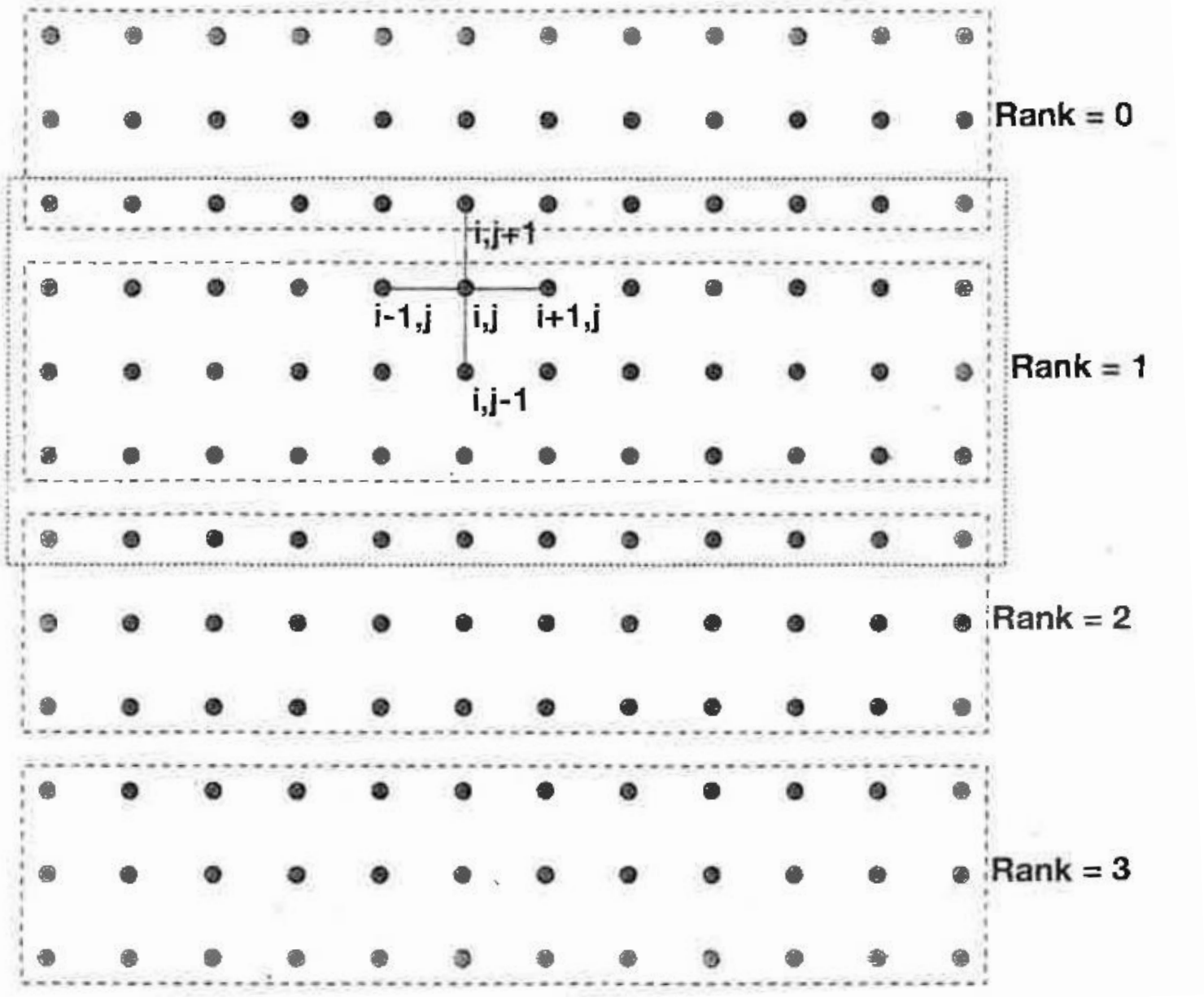
Isosurface Extraction Examples



Source: researchgate.net



Source: wisc.edu

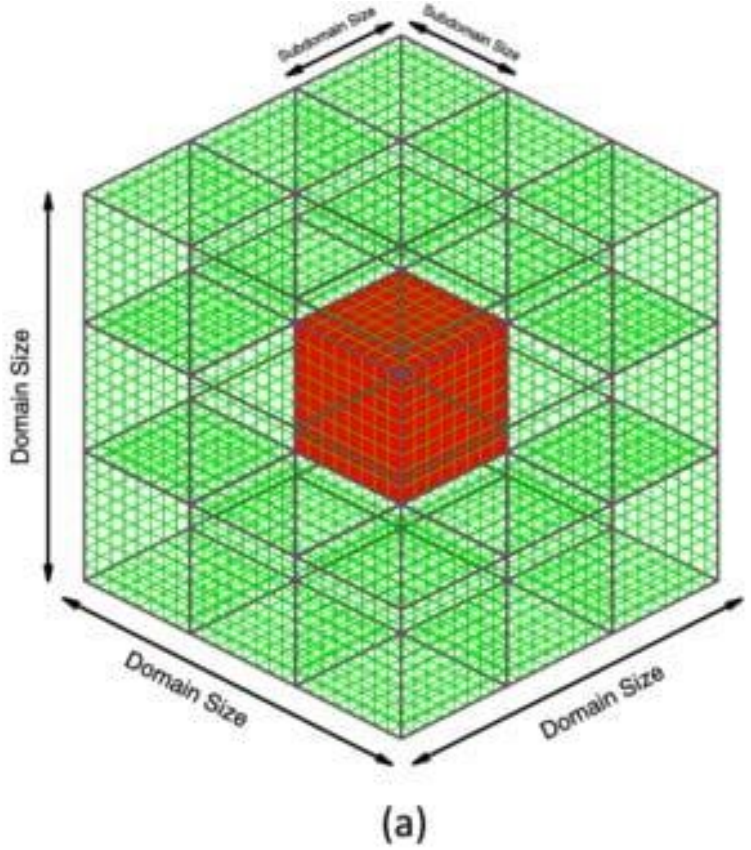


5-point stencil example

Field value at (i,j) for T+1th time step is calculated using the average of field values of itself and its E, W, N, S neighbours at the Tth step (i.e. sum of these five values/5)

Using Advanced MPI, Gropp et al.

Assignment 2 Recap



- A number of data fields (variables) defined at each grid point of a 3D domain
- Halo exchange
- Compute d-point stencil
- Compute the count of isovalues (note that you do not have to extract the isosurface)

- Eventually, root should output the total count per field per time step of all processes